Agentic Text-to-SQL 项目汇报

> **建议阅读 / 汇报时长：3～5 分钟**
> GitHub：https://github.com/Lx6uo/agentic-text2sql-cpp

项目卡片	内容
💡 形态	C++20 命令行工具
🤖 模型	DeepSeek `deepseek-flash`
🗄️ 数据库	任意已有 SQLite
🔧 技术	CMake、libcurl、nlohmann/json
🖥️ 平台	Windows / macOS / Linux

1. 🧭 一句话介绍

用自然语言安全查询 SQLite 数据库。

用户提问后，大模型自主查看表结构、生成只读 SQL、执行查询；SQL 出错时，根据真实错误有限修正。

```
“平均工资最高的部门？”
            ↓
查看表 → 查看字段 → 生成 SQL → 安全执行 → 返回结果
                                  ↘ 出错 → 修正
```

无需训练模型、无需本地模型、无需数据库服务器。

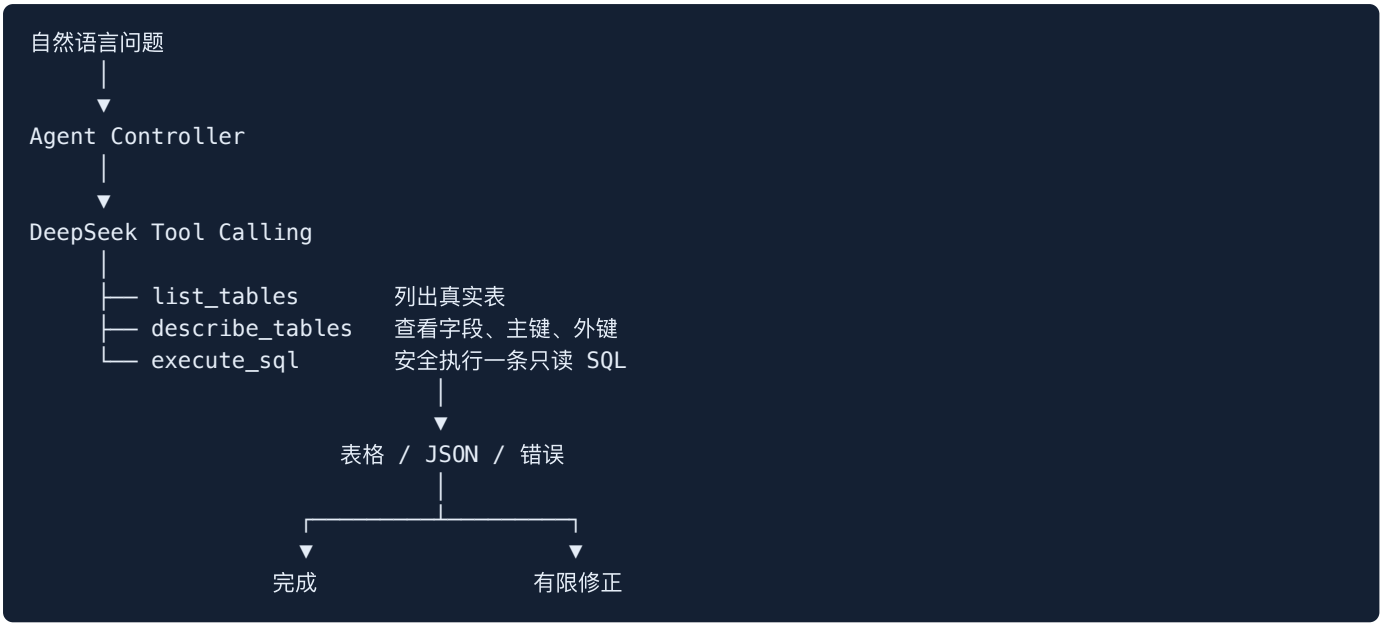
2. 🎯 为什么做

- **SQL 有门槛**：需要了解语法、表名、字段和关联关系。
- **陌生库难使用**：换一个数据库，就要重新阅读 Schema。
- **直接生成有风险**：字段幻觉、错误 SQL、写入或删除操作。

项目思路：

> **模型负责理解与规划，C++ / SQLite 负责权限与执行。**

3. 🧠 核心流程



默认限制：**最多 8 轮、3 次 SQL、2 次修正**，避免无限循环。

4. 🧰 已实现能力

能力	说明
💬 REPL	连续输入问题；`/tables`、`/schema`、`/open`
⚡ 单次查询	`query --db ... --question ...`
🔍 Verbose	展示 Agent 每一步工具调用
📦 JSON	stdout 仅输出机器可读 JSON
🎮 离线 Demo	无 Key、无网络，SQLite 仍真实执行
📊 Spider	Baseline / Agentic 对照、JSONL 与预测 SQL 导出

5. 🔒 安全设计

不是简单检查 SQL 是否以 `SELECT` 开头，而是 SQLite 多层强制：

- `SQLITE_OPEN_READONLY`：数据库只读打开。
- `sqlite3_stmt_readonly`：拒绝非只读语句。
- `sqlite3_set_authorizer`：拒绝写入、DDL、ATTACH、危险 PRAGMA、未知函数。
- 拒绝多条 SQL；禁用扩展加载。
- 查询超时、行数、SQL 长度、结果大小限制。
- Ctrl+C 可取消。

```
LLM 生成 SQL ≠ 获得数据库权限
最终是否执行，由 C++ 安全层决定
```

6. 💻 体验示范

6.1 离线错误修正

直接复制：

```
./build/text2sql demo --repair --verbose
```

核心输出：

```
[1] list_tables: Success
[2] describe_tables: Success
[3] execute_sql: Failed — no such column: department_name
[4] execute_sql: Success

Results:
department   average_salary
Engineering  115000.0
Finance      95000.0
Marketing    82500.0

Agent steps: 5, SQL attempts: 2, repairs: 1
```

含义： 第一次 SQL 使用错误字段；SQLite 返回真实错误；第二次修正成功。

6.2 真实自然语言查询

需先填写 `config.json`。示例数据库不存在时，先创建：

```
test -f data/company.sqlite || sqlite3 data/company.sqlite < examples/demo.sql
```

启动交互式命令行，并直接输入自然语言：

```
$ ./build/text2sql --db data/company.sqlite --verbose
Agentic Text-to-SQL
Model: deepseek-flash
Type /help for commands.

> 查询平均工资最高的三个部门，返回部门名称、平均工资和员工人数
[1] list_tables: Success
[2] describe_tables: Success
[3] execute_sql: Success

SQL:
SELECT d.name AS 部门名称, AVG(e.salary) AS 平均工资,
       COUNT(e.id) AS 员工人数
FROM employees e
JOIN departments d ON e.department_id = d.id
GROUP BY d.id, d.name
ORDER BY 平均工资 DESC
LIMIT 3

Results:
部门名称      平均工资   员工人数
Engineering  115000.0  2
Finance      95000.0   2
Marketing    82500.0   2
3 row(s)

Agent steps: 4, SQL attempts: 1, repairs: 0
>
```

> 真实模型生成的 SQL、措辞和耗时可能略有变化；数据库结果应一致。

7. 📊 项目评测结果

从 Spider dev 的 **20 个数据库各取 1 题**，覆盖 20 种不同 Schema：

项目性能指标	Agentic	Baseline
SQL 执行成功率	✅ 20 / 20（100%）	✅ 20 / 20（100%）
与 Gold SQL 结果一致	13 / 20（65%）	15 / 20（75%）
Agent 正常完成率	19 / 20（95%）	20 / 20（100%）
平均 Agent 步数	4.4	1.0
平均 SQL 尝试	1.35	1.0
平均响应耗时	4.23 s / 题	0.91 s / 题
平均 Token	约 5,644 / 题	约 1,168 / 题

```
每题平均流程：
list_tables → describe_tables → execute_sql → 最终回答
```

结论： Agent 可以跨 20 个数据库稳定生成可执行 SQL；当前结果一致率为 65%，低于 Baseline 的 75%。动态 Schema 探索提供了错误反馈和修正能力，但也增加了步骤、耗时和 token，语义精度仍是主要改进方向。

> 20 题用于跨库阶段性评估，不代表完整 1034 题 dev 准确率；本地结果一致率也不等于官方 Test Suite Accuracy。

8. ✨ 项目亮点

- **真实闭环**：Schema → SQL → 执行 → 错误 → 修正。
- **确定性安全**：模型建议，程序裁决。
- **跨数据库**：不写死 Demo 字段。
- **轻量工程**：无 LangChain、无 Web 服务、无多 Agent。
- **可离线演示**：Mock 模型 + 真实 SQLite。
- **跨平台支持**：Windows / macOS / Linux。

9. 🚀 改进方向

方向	计划能力	使用价值
🧠 多轮上下文	`/context on/off`、`/clear`、最近若干轮历史	支持“继续”“再筛选”等追问
🔄 上下文切换	`/open` 切库自动清理；上下文长度上限	避免旧 Schema 污染和 token 无限增长
📈 结果分析	统计摘要、分布、排名、异常值、分组比较	查询后直接获得数据洞察
📤 分析输出	文本 / JSON；预留图表与 CSV 导出	便于汇报和后续程序集成

数据分析交互示例：

```
> 查询每个部门的平均工资
> 分析刚才结果，指出最高值、差距和异常项
```

10. 🎞️ PPT 建议：8 页

页码	标题	关键信息 / 视觉
1	项目简介	一句话定位 + 终端截图
2	问题与目标	SQL 门槛、陌生 Schema、安全风险
3	系统架构	自然语言 → Agent → 三工具 → SQLite
4	Agentic 流程	探索、执行、报错、修正时间线
5	安全设计	多层只读盾牌图
6	运行演示	本文第 6 节输入输出
7	项目评测结果	20 库、100% 可执行、65% 结果一致率、成本对照
8	亮点与改进	安全解耦、上下文切换、结果分析、总结

11. ✅ 总结

> Agentic Text-to-SQL 不是让模型“猜一条 SQL”，而是让模型在**安全、有限、可观测**的工具环境中完成数据库查询。

当前状态：**可构建、可运行、可演示、可测试、可跨平台、可做公开数据集实验。**